Predicting English Premier League Match Outcomes

Logistic Regression from Scratch with Gradient Descent

Course: Machine Learning | **University:** USTH | April 2026

Nghiêm Trọng Quốc Anh	23BA14004	Lê Đức Anh	23BA14005
Lương Quỳnh Anh	23BA14008	Nguyễn Hải Anh	23BA14002
Nguyễn Cao Việt Anh	23BA14009		

AGENDA

🎯 1. Foundation

- Problem Statement
- Algorithm Theory
- L2 & Normalization

📊 2. Data & Code

- Dataset & Source
- Feature Engineering
- Evaluation Metrics
- `model.py` Walkthrough

🚀 3. Results

- Pipeline Workflow
- Improvements & Results
- Future Work
- Conclusion

PROBLEM STATEMENT

📌 Classification Task

Binary Classification — predict whether the home team **wins (1)** or **does not win (0)**

⚙️ Core Constraints

- Build pure ML model **from scratch**
- Use **NumPy only** (No scikit-learn)
- Self-implemented Gradient Descent

⚠️ Challenges

- Football is inherently unpredictable
- Data noise and class imbalance

Pre-Match Features

↓

Logistic Regression

↓

WIN (1) **NOT WIN (0)**

ALGORITHM: LOGISTIC REGRESSION

1. Linear Score (*z*)

$$z = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \theta^T x$$

Combines features (*x*) with learned weights (θ)

2. Sigmoid Activation (Probability)

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Squashes the score *z* into probability range [0, 1]

3. Log Loss (Cost Function *J*)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Penalizes wrong predictions exponentially

4. Gradient Descent (Update)

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

α = learning rate, iteratively minimizes cost

Decision Rule: IF ŷ ≥ **threshold** → **Predict Win (1)** | ELSE → **Predict Not Win (0)**

L2 REGULARIZATION & NORMALIZATION

L2 Regularization

Adds penalty for **large weights** to prevent model from memorizing noise (overfitting).

$$J_{reg}(\theta) = J(\theta) + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

- Controls model complexity
- **λ (lambda)** = regularization strength
- Gradient updated: `dw += (λ/m) · w`

Min-Max Normalization

Scales disparate features into the same standard range **[0, 1]**.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Crucial:** Parameters fit on **Train set** only!
- Applied symmetrically to Test set
- Ensures stable and fast Gradient Descent

DATASET & PREPARATION

🌐 Data Source

- **football-data.co.uk** (official EPL results)
- Aggregated over multiple seasons (**2017/18 → 2025/26**)

📊 Raw Columns

Variable	Description
`Date`, `Teams`	Match identifiers
`FTR`	Full-Time Result (H/D/A)
`FTHG/FTAG`	Goals scored
`B365H, PSH`	Live betting odds

🔀 Train / Test Split Model

Mode: Season-Based Split

Train: All historical seasons

Test: Latest season **2025/26** (~1,430 matches)

Target Label Mapping

```
target = 1 ← FTR = "H"
target = 0 ← FTR = "D" | "A"
```

*** Simulates real-world robust scenario**

FEATURE ENGINEERING (8 INPUTS)

Feature	Math Formula / Logic	Intuitive Meaning
`RankDiff`	rank_away – rank_home	League standings gap
`WinRateDiff`	W/Played (home) – (away)	Historical win efficiency
`PPGDiff`	Pts/Played (home) – (away)	Points-per-game difference
`ELODiff`	elo_h – elo_a (K=20)	Dynamic strength rating
`RecentFormDiff`	sum(last 3: W=1,D=0,L=-1)	Short-term momentum
`RestDaysDiff`	rest_days_home – away	Physical fatigue factor
`MarketProbDiff`	(1/odd_h) – (1/odd_a)	Bookmaker aggregated wisdom
`HomeAdvantage`	constant = 1	Bias for home stadium

🛡️ **Strict Anti-Leakage Policy**

All features are computed **STRICTLY BEFORE** kickoff using cumulative history arrays.

EVALUATION METRICS

Metric Overview	Formula
Accuracy	**TP + TN** / Total
Precision	TP / (**TP + FP**)
Recall	TP / (**TP + FN**)
F1 Score	2PR / (P + R)

⚖️ The Threshold Trade-off

Adjusting the decision boundary changes everything:

- **↑ High Threshold = Better Precision, Worse Recall**
- **↓ Low Threshold = Better Recall, Worse Precision**

Confusion Matrix

	Pred: Win	Pred: Not Win
Actual: Win	**TP**	**FN**
Actual: Not Win	**FP**	**TN**

Precision: "When we yell WIN, how often are we right?"

Recall: "Of all actual WINS, how many did we catch?"

CODE ARCHITECTURE

```
project-machine-learning/
├── data/
│   ├── data-raw/              # Raw EPL CSVs
│   └── epl_real_features.csv
├── src/
│   ├── prepare_real_data.py   # Gen features
│   ├── preprocessing.py       # MinMax Setup
│   ├── model.py               # Core Logic
│   ├── metrics.py             # Evaluation
│   ├── train.py               # Training loop
│   ├── evaluate_thresholds.py # Sweeper
│   └── tune_hyperparams.py    # Grid searcher
├── outputs/
├── Dockerfile
└── docker-compose.yml
```

🔑 Key Design Pattern

Highly modularized scripts. Object-Oriented wrapping for Models and Scalers.

📂 Separation of Concerns

- **Data Prep** is decoupled from **Training**.
- **Tuning Grid** relies on reusable blocks.

🐳 Infrastructure

Packaged standardly:
`docker compose run --rm epl`

ALGORITHM IMPLEMENTATION (MODEL.PY)

1. Sigmoid Function

```
@staticmethod
def _sigmoid(z):
    z = np.clip(z, -500, 500) # No overflow
    return 1.0 / (1.0 + np.exp(-z))
```

2. Log Loss Calculation

```
@staticmethod
def _log_loss(y_true, y_pred):
    eps = 1e-12
    p = np.clip(y_pred, eps, 1-eps) # No log(0)
    return -np.mean(
      y_true * np.log(p) + (1-y_true) * np.log(1-p)
    )
```

3. Gradient Descent Loop + L2

```
def fit(self, x, y):
    # Init weights
    self.w, self.b = np.zeros(x.shape[1]), 0.0

    for _ in range(self.epochs):
      # Forward probs
      p = self._sigmoid(np.dot(x, self.w) + self.b)

      # Raw gradients
      dw = (1/m) * np.dot(x.T, (p - y))
      db = (1/m) * np.sum(p - y)

      # Apply L2
      if self.l2 > 0: dw += (self.l2/m) * self.w

      # Update
      self.w -= self.lr * dw
      self.b -= self.lr * db
```

END-TO-END WORKFLOW

1

Data Ingestion

`football-data.co.uk`

- Parse dates & std names
- Accumulate form & Elo
- `epl_real_features.csv`

→

2

Training Focus

`train.py`

- Season-based split
- Min-Max fit on Train
- Gradient Descent loop

→

3

Optimization

`tune_hyperparams.py`

- Sweep Thresholds
- 180 Config Grid Search
- **Find Best Accuracy**

Step 4: Output Generation

Generated artifacts: `model_weights.json`, `metrics.json`, `predictions.csv`

IMPROVEMENTS & FINAL RESULTS

📈 Development Journey

Version	Key Action Taken
V1. BASE	Minimal features, threshold = 0.5
V2. FEAT	Injected Elo + Bookmaker odds
V3. THRESH	Swept threshold curve → optimized recall
V4. TUNE	180 configs Grid Search
V5. FINAL	Season strict split (Test on 25/26)

⭐ Top 3 Vital Features

Feature	Learned Weight
MarketImpliedProbDiff	**2.755** (Dominant)
RankDiff	**0.696**
ELODiff	0.325

🔥 Test Set Performance (Season 25/26)

65.94%
MASTER ACCURACY

0.608
LOG LOSS TARGET

63.0%
PRECISION

56.1%
RECALL

FUTURE DEVELOPMENT

1. Advanced Stats

- Incorporate **xG** (Expected Goals) metrics.
- Add Match Possession & Shots on Target stats.
- Capture hidden team qualities.

IMPACT: HIGH

2. Class Balancing

- Integrate **Class Weights** directly into Cost Function.
- Address structural label imbalance natively.
- Boosts minority class recall.

IMPACT: MEDIUM

3. Smart Search

- Replace brute Grid Search with **Bayesian Optimization**.
- Much faster parameter convergence.
- Enables searching continuous spaces.

IMPACT: HIGH EFFICIENCY

*** All keep Logistic Regression as core — purely improving Data Signal & Optimization Strategy.**

CONCLUSION

✓ What We Achieved

- Built mathematical foundation **completely from scratch** (No APIs).
- Engineered 8 complex contextual pre-match features.
- Achieved rigid testing **Accuracy of 65.94%**
- Systematic & reproducible pipeline.

💡 Key Takeaways

Even a fundamental algorithm like Logistic Regression achieves **highly competitive real-world bounds** when:

- **Features isolate the right signals.**
- **Regularization is applied.**
- **Evaluation mirrors reality.**

🚀

Thank you for listening! Questions?